

Práctica 8: BIM semántico

~8 min de lectura

Objetivos de la práctica

En esta práctica trabajaremos con un mismo edificio en dos representaciones complementarias: un modelo BIM en formato IFC y una versión semántica en OWL. Al finalizar, deberías ser capaz de:

- Entender qué aporta BIM como sistema de organización de información de un edificio.
- Explorar un modelo IFC con un visor BIM y localizar elementos concretos.
- Abrir una ontología en *Protégé* y recorrer sus clases, propiedades e individuos.
- Realizar consultas semánticas con un razonador para recuperar información del modelo.

Introducción a BIM

El valor de BIM se aprecia mejor cuando pensamos en el **ciclo de vida** completo de la edificación. La misma información puede servir para coordinar el proyecto, detectar interferencias, planificar fases de obra o consultar características de un elemento una vez construido.

Idea clave

En esta práctica no vamos a modelar un edificio desde cero. Vamos a aprender a **leer** un modelo BIM y a **consultar** su información, primero con un visor IFC y después con herramientas semánticas.

Material de trabajo

Utilizaremos los siguientes ficheros:

- [Duplex.ifc](#), que contiene el modelo BIM original del apartamento dúplex en formato IFC.

El **Modelado de Información de Construcción (BIM, Building Information Modeling)** es una forma de representar digitalmente un edificio y la información asociada a sus elementos. No se trata solo de una maqueta 3D: un modelo BIM también puede describir materiales, relaciones espaciales, identificadores, propiedades geométricas y datos útiles para diseño, obra, mantenimiento y explotación. Esta manera de trabajar resulta especialmente valiosa porque reúne en un mismo modelo información que normalmente estaría dispersa en planos, tablas, memorias y documentos técnicos. Por eso BIM se ha convertido en una pieza clave en la digitalización del sector de la construcción.



Figura 1: Esquema general del flujo de trabajo BIM.

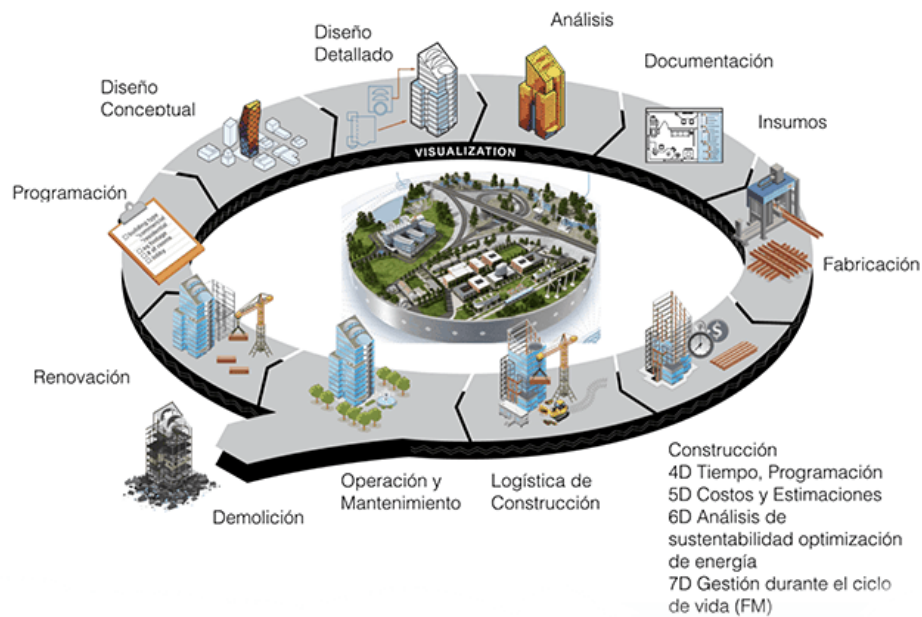


Figura 2: Ciclo de vida de la edificación y papel del modelo BIM.

- [Duplex.owl](#), que contiene una versión semántica del mismo modelo en lenguaje **OWL**.
- [beo.owl](#) y [bot.owl](#), que pueden ser necesarios si *Protégé* te pide resolver manualmente algunas importaciones.

Como software de apoyo, se recomienda utilizar [BIMvision](#) para visualizar el IFC y [Protégé](#) para explorar la ontología.

Ejercicio 1: explorar el modelo IFC

El primer paso es abrir [Duplex.ifc](#) en **BIMvision** o en cualquier otro visor BIM compatible con IFC. El objetivo no es todavía hacer consultas complejas, sino familiarizarse con el edificio, sus plantas y la forma en que cada elemento aparece identificado en el modelo.

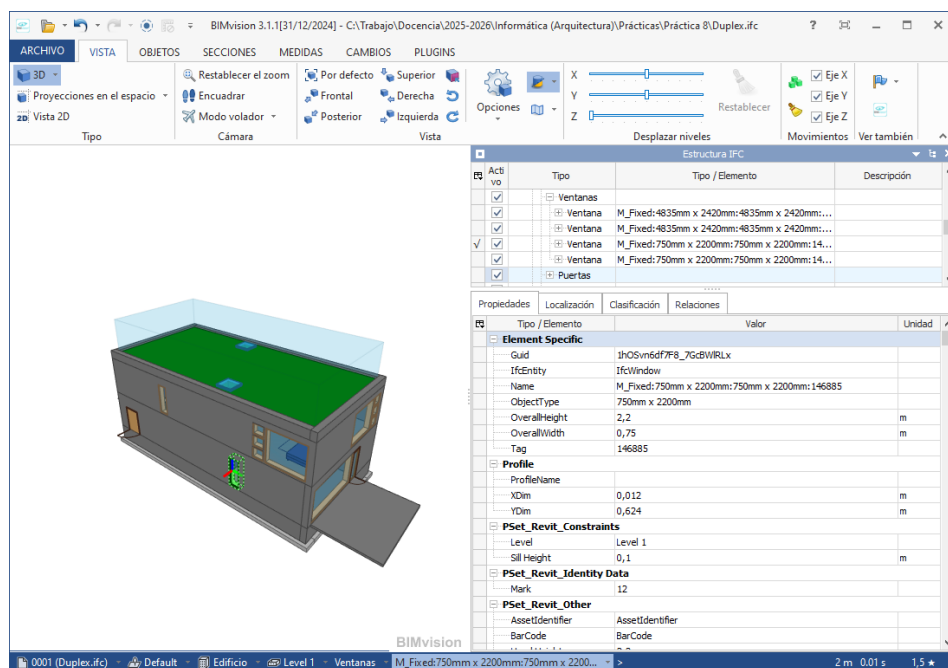


Figura 3: Modelo Duplex abierto en BIMvision.

💡 Qué debes observar

Empieza por recorrer la planta baja y la planta superior, y acostúmbrate a seleccionar elementos individuales. Fíjate en cómo el visor muestra sus nombres, identificadores y propiedades básicas.

Durante esta primera exploración, responde a las siguientes preguntas:

- ¿Cuántas ventanas hay en el modelo?

- ¿Qué identificador tiene cada una?
- ¿Qué altura y qué anchura tiene cada ventana?

Del IFC al modelo semántico

El siguiente paso consiste en pasar de una representación BIM convencional a una representación **semántica**. En lugar de trabajar solamente con geometría y propiedades dentro de un visor, vamos a describir el edificio mediante clases, propiedades e individuos conectados entre sí en una ontología.

En esta práctica no es necesario que hagas la conversión por tu cuenta, porque el fichero `Duplex.owl` ya está preparado. Aun así, conviene entender de dónde sale: se ha generado a partir del IFC usando [IFCtoLBD 2.43.5](#), una herramienta que transforma modelos IFC en datos enlazados reutilizando vocabularios existentes del ámbito de la construcción.

Conversión utilizada

El fichero `Duplex.owl` se ha generado con la configuración **BOT + PRODUCT + PROPS, Level L1**. Además, se han eliminado manualmente las referencias a dos ontologías externas para simplificar la práctica.

Ejercicio 2: abrir la ontología en Protégé

Para abrir `Duplex.owl` usaremos **Protégé**, un editor de ontologías gratuito y disponible para varios sistemas operativos. No suele requerir instalación compleja: basta con descargar el archivo comprimido, descomprimirlo y ejecutar la aplicación.

Si faltan importaciones

En algunas versiones de *Protégé*, las ontologías importadas se descargan automáticamente desde Internet. En otras, tendrás que añadirlas manualmente desde **Active ontology > Ontology imports** usando los ficheros `beo.owl` y `bot.owl`.

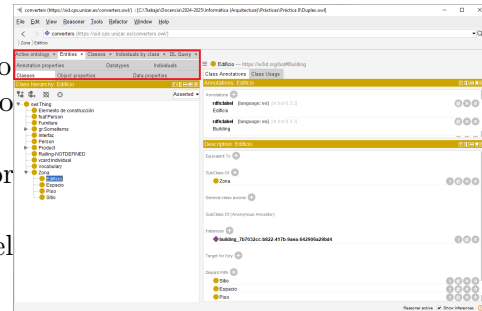
También debes tener en cuenta que muchas entidades tienen **etiquetas en varios idiomas**. Dependiendo de la configuración de tu equipo, *Protégé* puede mostrar nombres en castellano o en inglés. En este guion se asume que las etiquetas se muestran en castellano siempre que sea posible.

Qué debes localizar

Dentro de la ontología, localiza:

Una vez abierto el fichero, encontrarás varias vistas desde las que puedes navegar por la ontología. En el menú lateral de **Entities** podrás recorrer:

- **Classes**, para ver la jerarquía de clases del dominio
- **Object properties**, para ver propiedades cuyo valor es otro objeto o individuo.
- **Data properties**, para ver propiedades cuyo valor es un dato literal.
- **Individuals**, para ver las instancias concretas del modelo.



La interfaz puede cambiar ligeramente según la versión de *Protégé*, pero la idea general es siempre la misma: pasar de la estructura conceptual a los elementos concretos del edificio.

Figura 4: Jerarquía de clases y detalle de una clase en *Protégé*.

- La clase que permite representar las ventanas.
- Las propiedades que representan la altura y la anchura de una ventana.
- Una ventana concreta y los valores de sus propiedades.

Comprueba además si esos valores coinciden con los que viste en el visor IFC durante el ejercicio anterior.

Solución orientativa: clases

Solución orientativa: propiedades

Solución orientativa: individuos

Ejercicio 3: razonamiento e inferencia

Una de las ventajas de trabajar con ontologías es que no solo podemos leer datos explícitos, sino también **inferir** información nueva a partir de la estructura del modelo y de las restricciones definidas en la ontología.

Para ello, activa un razonador desde el menú superior **Reasoner**. En esta práctica se recomienda utilizar **HermiT**. Una vez cargado, podrás abrir la pestaña **DL Query** para lanzar consultas sobre clases e individuos.

Consultas propuestas

Con el razonador activo, realiza las siguientes comprobaciones:

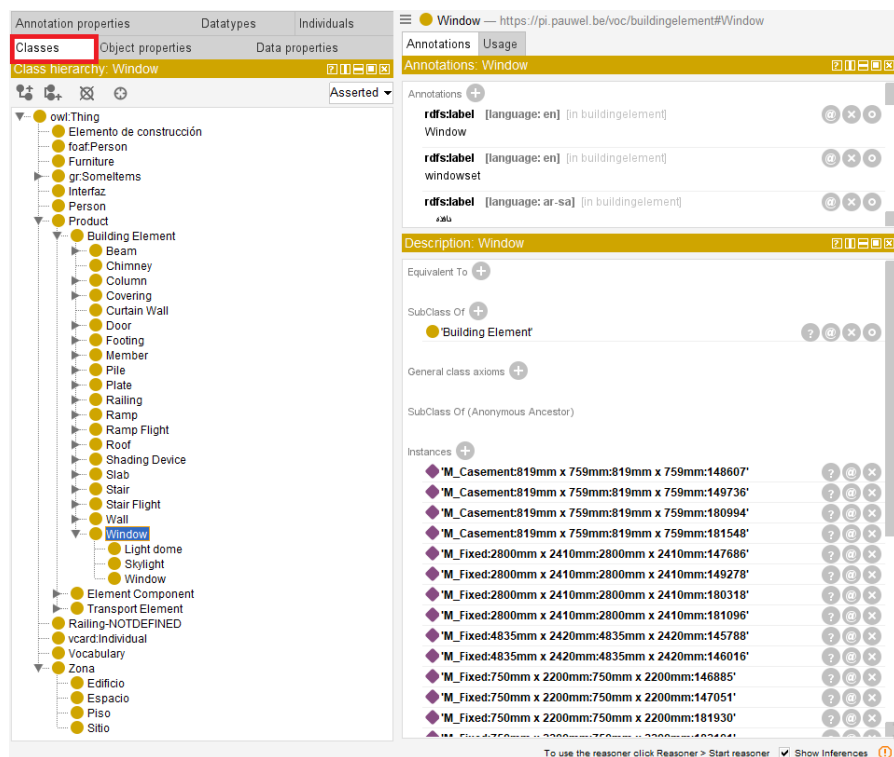


Figura 5: Jerarquía de clases para localizar la entidad de ventana.

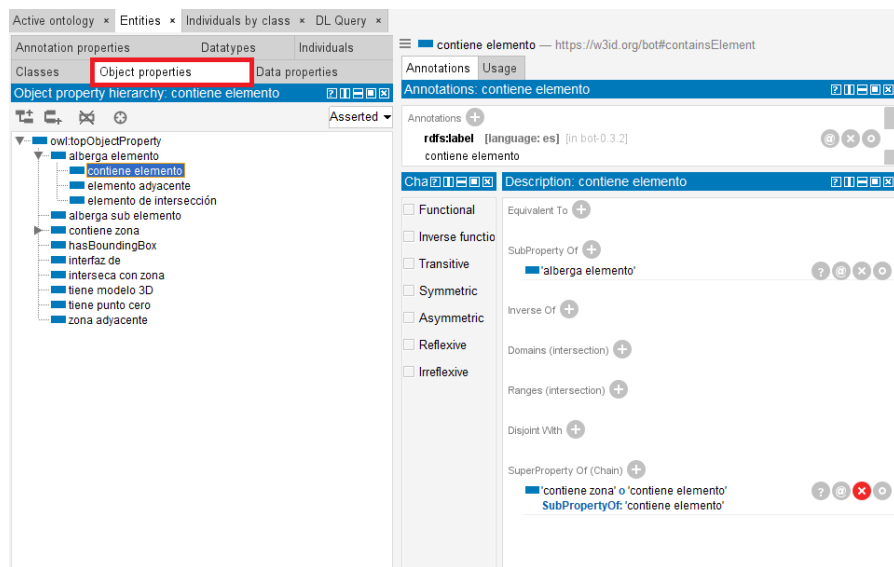


Figura 6: Propiedades de objeto relacionadas con los elementos del modelo.

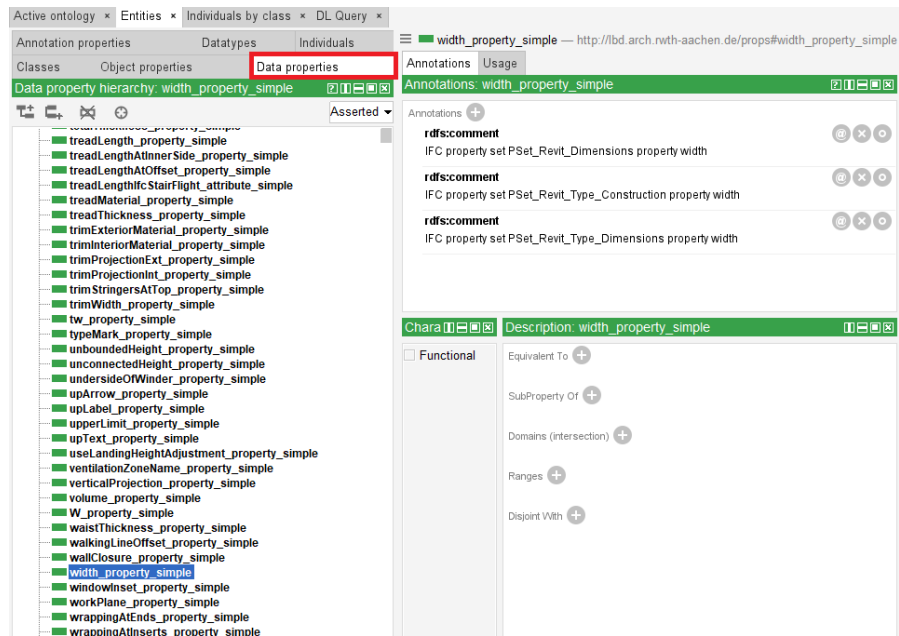


Figura 7: Propiedades de datos para recuperar medidas y otros valores literales.

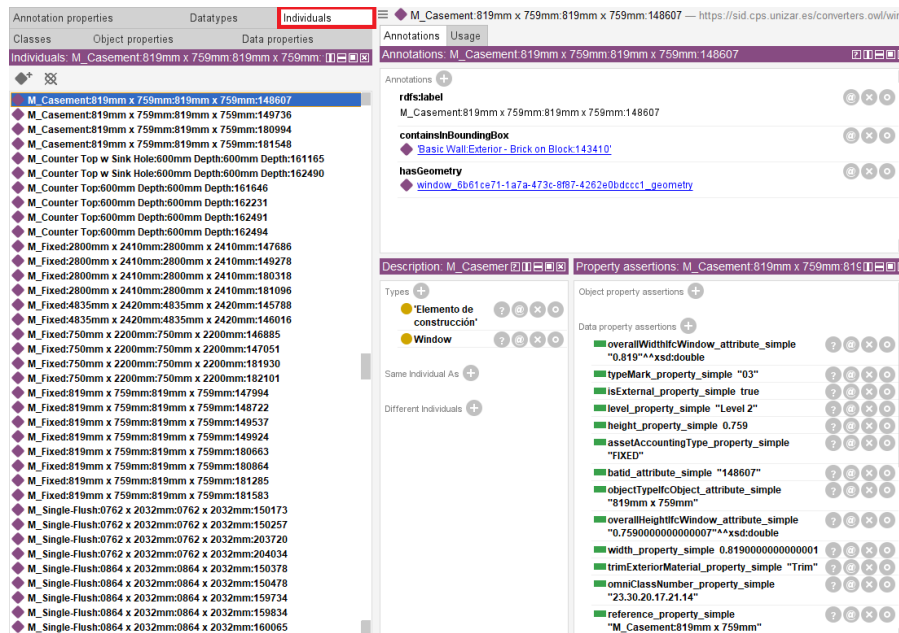


Figura 8: Exploración de individuos concretos dentro de la ontología.

- Recupera las instancias de la clase que representa las **ventanas**.
- Recupera las **superclases** de esa clase.
- Recupera las instancias de alguna de esas superclases para comprobar si el razonador también devuelve instancias indirectas.
- Vuelve a consultar las instancias de la clase de ventanas y haz clic en el símbolo ? de alguna de ellas para ver al menos una explicación.

i Pista

En muchas instalaciones, las consultas se escriben con los nombres cortos en inglés, por ejemplo **Window**, aunque en la interfaz veas etiquetas en castellano.

Ejemplo orientativo: consulta sobre escaleras

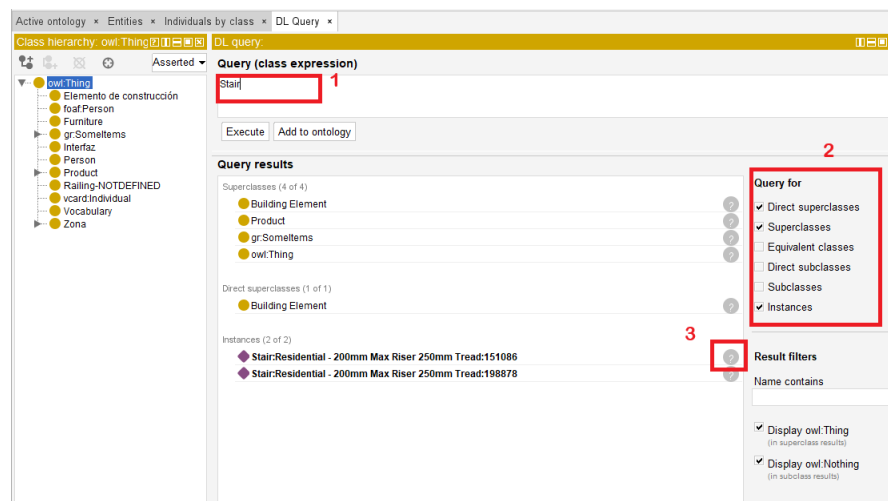


Figura 9: Ejemplo de consulta DL en Protégé.

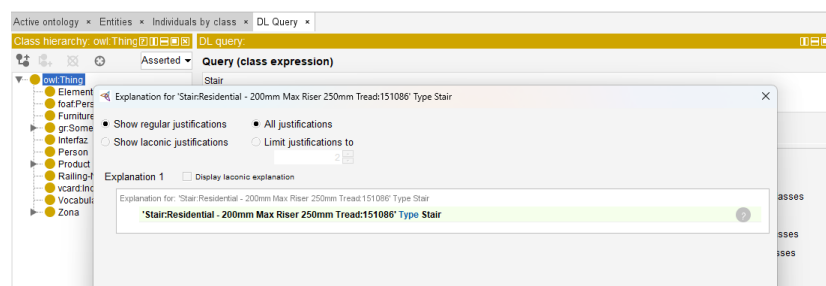


Figura 10: Ejemplo de explicación ofrecida por el razonador.

Ejercicio 4: consultas más expresivas

Una vez dominadas las consultas básicas, podemos construir expresiones más ricas combinando clases y propiedades. Esto permite formular preguntas que no se reducen a “dame todas las instancias de una clase”, sino a “dame todos los elementos que cumplan una determinada condición”.

Lanza la siguiente consulta:

```
Espacio and 'elemento adyacente' some Window
```

Esa expresión representa el conjunto de espacios que tienen al menos un elemento adyacente que pertenece a la clase **Window**.

Responde a estas preguntas:

- ¿Cuántos individuos recupera la consulta?
- ¿Qué tipo de espacios son?
- ¿Coincide ese resultado con lo que esperarías al observar el edificio en el visor IFC?

Ejercicio 5: provocar una inconsistencia

Si una ontología se vuelve **inconsistente**, el razonador ya no puede trabajar correctamente con ella. En ese caso, *Protégé* muestra un aviso y permite abrir una explicación para entender qué axiomas están entrando en conflicto.

En este ejercicio vamos a provocar una inconsistencia de forma controlada. La idea es declarar como **funcional** la propiedad que representa la altura de una ventana y, después, asignar a una misma ventana dos valores distintos para esa altura.

Pasos sugeridos

1. Localiza la propiedad de datos que representa la altura.
2. Márcala como **Functional**.
3. Busca una ventana concreta en **Individuals**.
4. Añade una segunda afirmación de tipo **Data property assertion** con un valor distinto del original, por ejemplo 1234.
5. Sincroniza el razonador y observa el error de inconsistencia.

Recomendación

Haz este ejercicio sobre una copia de trabajo del fichero si quieres conservar **Duplex.owl** sin cambios al terminar la práctica.

Solución orientativa: inconsistencia

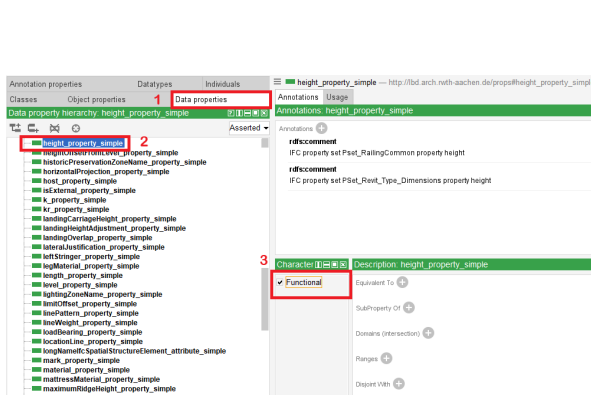


Figura 11: Marcar la propiedad como funcional.

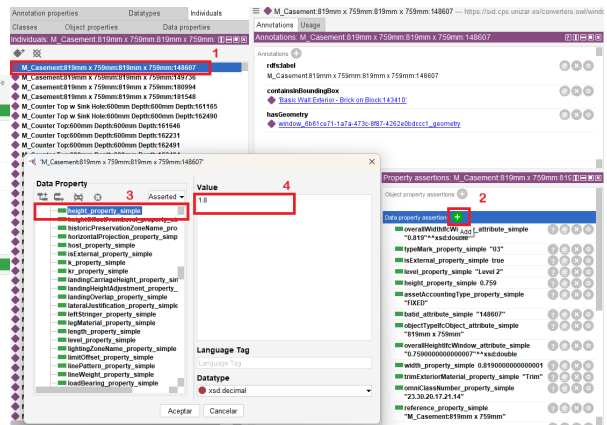


Figura 12: Añadir un segundo valor incompatible para la propiedad.

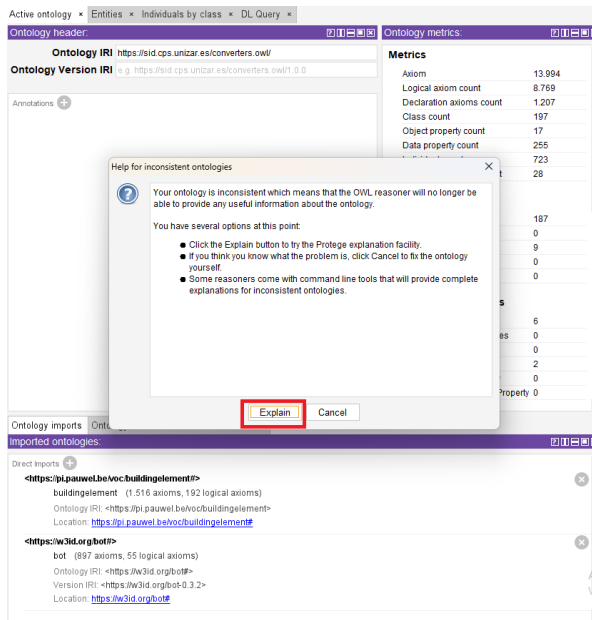


Figura 13: Aviso de inconsistencia y acceso a la explicación.

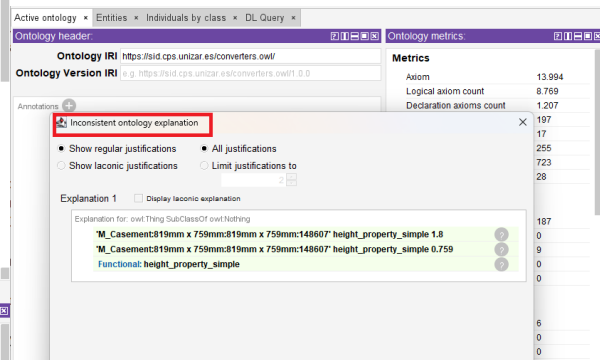


Figura 14: Explicación de la inconsistencia generada.